

# Plano: Faturas — Import de Arquivo + Proteção de Muta  o

## Context

Two PRs to harden the invoice/ledger domain:

**PR 1** replaces the fragile "recognize invoice on payment" flow with an atomic import that creates invoice + expenses + fires `INVOICE_RECOGNIZED` in one transaction, using the total known at import time. The card's `paymentAccountId` (which account pays the bill) is the routing key for the ledger event.

**PR 2** guards `UpdateExpense` / `UpdateRevenue` against changing the bank account after a financial event has already been recorded — which would silently diverge the ledger.

---

## Preliminary: No Reverts Needed

The current `master` branch already has the clean state (no `FinancialEventsService` in `UpdateStatusInvoice`, no `accountId` in `updateInvoiceStatus`, no `FinancialEventsModule` in `InvoicesModule`). The "revert" section in the draft refers to a separate `gateways` branch and can be skipped.

---

## PR 1 — Import de Faturas via Arquivo

### Architecture

```
POST /invoices/import (multipart: file + idCreditCard)
|
▼
ImportInvoiceUseCase
├─ CreditCard loaded via InvoicesRepository.getCreditCardForImport(id)
├─ Validate paymentAccountId > 0
├─ InvoiceParserFactory.create(mimetype) → IInvoiceFileParser
├─ parser.parse(buffer) → InvoiceImportRowDto[]
├─ Derive dueDate per row (Luxon, closingDay + dueDay algorithm)
├─ Group rows by dueDate
└─ database.transaction():
    For each (dueDate → rows):
      a. findOrCreateInvoice(creditCard, closingDateISO, dueDateISO)
      b. Create Expense per row (idCreditCard, idInvoice, idBankAccount=0, pending)
      c. FinancialEventsService.register({ accountId: paymentAccountId, ...
INVOICE_RECOGNIZED })
      d. LedgerEngine.process(event)
    Return InvoiceImportResultDto
```

### Step 1 — DB Migration: Add `INVOICE_RECOGNIZED` to `financial_events` ENUM

**New file:** `src/migrations/versions/Migration20260410_01_AdicionaInvoiceRecognized.ts`

```
ALTER TABLE financial_events
  MODIFY COLUMN type ENUM(
    'PAYMENT_POSTED', 'TRANSFER_POSTED', 'REVERSAL',
    'OPENING_BALANCE', 'REVENUE_RECEIVED', 'INVOICE_RECOGNIZED'
  ) NOT NULL;
```

The `type` column in `financial_events` is a MySQL `ENUM` (see `Migration20260228_01`). Adding a new value requires an `ALTER`.

## Step 2 — Enum + Ledger Builder

`src/enum/financial-events.enum.ts` — add:

```
INVOICE_RECOGNIZED = 'INVOICE_RECOGNIZED'
```

`src/modules/Finance/core/ledger/ledger.builder.ts` — add case in `defineAmounts()` :

```
case FinancialEventsEnum.INVOICE_RECOGNIZED:
  return { originAmount: Math.abs(event.amount), destinationAmount: -
    Math.abs(event.amount) };
```

Same direction as `PAYMENT_POSTED` (liability is recognized, earmarks the payment account).

## Step 3 — DB Migration: `paymentAccountId` on `credit_cards`

New file:

`src/migrations/versions/Migration20260410_02_AdicionaPaymentAccountIdCreditCard.ts`

```
ALTER TABLE credit_cards ADD COLUMN paymentAccountId BIGINT NULL DEFAULT NULL;
```

## Step 4 — CreditCard entity/interface/DTO

`src/EntityModels/CreditCard.ts`

- Add field: `paymentAccountId: number = 0`
- Update `fromDTO()` : `creditCard.paymentAccountId = dto.paymentAccountId ?? 0`
- Update `fromRow()` : `creditCard.paymentAccountId = row.paymentAccountId ?? 0`
- `getIgnoredProperties()` stays empty (`paymentAccountId` must be persisted)

`src/Shared/interfaces/ICreditCardRow.ts` — add: `paymentAccountId?: number`

`src/modules/Finance/credit-cards/DTOs/credit-card.dto.ts` — add:

```
@IsOptional()
@IsInt({ message: 'Invalid paymentAccountId!' })
paymentAccountId?: number;
```

## Step 5 — InvoicesRepository: new helper methods

`src/modules/Finance/Invoices/invoices.repository.ts` — add two methods:

```
// Loads the credit card with paymentAccountId for the import flow.
// Avoids adding CreditCardsModule dependency to InvoicesModule.
async getCreditCardForImport(idCreditCard: number): Promise<CreditCard | null> {
  const query = `SELECT id, name, closingDate, dueDate, paymentAccountId
    FROM credit_cards WHERE clientId = ? AND id = ? LIMIT 1`;
  const result = await this.database.select(query,
    [this.authContext.getClientId(), idCreditCard]);
  return result[0] ? CreditCard.fromRow(result[0]) : null;
}

// Finds an invoice by credit card and exact dueDate (YYYY-MM-DD).
async findInvoiceByDueDate(idBankAccount: number, dueDate: string): Promise<Invoice
| null> {
  const query = `SELECT * FROM invoices WHERE clientId = ? AND idBankAccount = ?
AND dueDate = ? LIMIT 1`;
  const result = await this.database.select(query,
    [this.authContext.getClientId(), idBankAccount, dueDate]);
  return this.extractToEntity(result, Invoice)[0] ?? null;
}
```

## Step 6 — Install dependencies

```
npm install csv-parse
npm install --save-dev @types/multer
```

Neither `csv-parse` nor `@types/multer` are in `package.json`. `multer` itself ships bundled with `@nestjs/platform-express` (already in deps).

## Step 7 — Import module files

`src/modules/Finance/Invoices/import/dtos/invoice-import-request.dto.ts`

```
class InvoiceImportRequestDto {
  @IsInt() idCreditCard: number;
}
```

`src/modules/Finance/Invoices/import/dtos/invoice-import-row.dto.ts`

```
class InvoiceImportRowDto {
  description: string;
  amount: number; // positive = expense
  date: string; // YYYY-MM-DD
  category?: string;
}
```

`src/modules/Finance/Invoices/import/dtos/invoice-import-result.dto.ts`

```
class InvoiceImportResultDto {
  invoicesCreated: number;
}
```

```

    expensesCreated: number;
    errors: string[];
}

```

**src/modules/Finance/Invoices/import/parsers/invoice-file-parser.interface.ts**

```

interface IInvoiceFileParser {
    parse(buffer: Buffer): InvoiceImportRowDto[];
}

```

**src/modules/Finance/Invoices/import/parsers/csv.parser.ts**

- Use `csv-parse/sync` to parse buffer
- Expected columns: `date,description,amount` (+ optional `category` )
- Validate ISO date format and numeric amount
- Throw descriptive error on malformed rows

**src/modules/Finance/Invoices/import/parsers/ofx.parser.ts**

- Regex extraction of `<STMTTRN>` blocks
- Extract: `<DTPOSTED>` → `YYYYMMDD[...]` → `YYYY-MM-DD` , `<TRNAMT>` (negative = expense, invert sign), `<NAME>` or `<MEMO>`
- Supports both SGML (no XML header) and OFX XML variants

**src/modules/Finance/Invoices/import/parsers/invoice-parser.factory.ts**

```

static create(mimetype: string, filename: string): IInvoiceFileParser {
    if (mimetype === 'text/csv' || filename.endsWith('.csv')) return new CsvParser();
    if (mimetype === 'application/x-ofx' || filename.endsWith('.ofx')) return new OfxParser();
    throw new Error(`Unsupported file type: ${mimetype}`);
}

```

**src/modules/Finance/Invoices/import/use-cases/import-invoice.use-case.ts**

Key logic — date derivation (Luxon is already a dependency):

```

private deriveDueDate(transactionDate: string, closingDay: number, dueDay: number):
{ closingDateISO: string, dueDateISO: string } {
    const txDt = DateTime.fromISO(transactionDate);
    let closingDt = txDt.set({ day: closingDay });
    if (txDt > closingDt) closingDt = closingDt.plus({ months: 1 });
    let dueDt = closingDt.set({ day: dueDay });
    if (dueDt <= closingDt) dueDt = dueDt.plus({ months: 1 });
    return {
        closingDateISO: closingDt.startOf('day').toISODate(),
        dueDateISO: dueDt.startOf('day').toISODate(),
    };
}

```

Full `execute()` flow:

1. Load CreditCard via `repository.getCreditCardForImport(idCreditCard)` → validate `paymentAccountId > 0`
2. Parse file via factory
3. Derive dueDate per row; group by `dueDateISO`
4. `database.transaction()` :
  - For each group:
    - `findOrCreate : repository.findInvoiceByDueDate(idCreditCard, dueDateISO) → if null, repository.createInvoice(new Invoice{ idBankAccount, name, closingDate, dueDate, status:'pending' })`
    - For each row: `expensesRepository.saveExpense(new Expense{ name:row.description, value:row.amount, idCreditCard, idInvoice, idBankAccount:0, status:'pending', linkToInvoice:true, invoiceDueDate:dueDateISO })`
    - Register event: `financialEventsService.register({ accountId: creditCard.paymentAccountId, amount: -total, type: INVOICE_RECOGNIZED, referenceType: PaymentType.INVOICE, referenceId: invoice.id })`
    - `ledgerEngine.process(event)`
5. Return `InvoiceImportResultDto`

*Amount for `INVOICE_RECOGNIZED` : pass as negative (expense direction); defineAmounts will apply `Math.abs` so the sign doesn't matter — but use negative to be consistent with `PAYMENT_POSTED` caller pattern in `PaymentService.normalizeAmount()` .*

## Step 8 — InvoicesController: add import endpoint

`src/modules/Finance/Invoices/invoices.controller.ts`

```
@Post('/import')
@HttpCode(HttpStatus.CREATED)
@UseInterceptors(FileInterceptor('file'))
async importInvoice(
  @UploadedFile() file: Express.Multer.File,
  @Body() body: InvoiceImportRequestDto,
) {
  return this.importInvoiceUseCase.execute(body.idCreditCard, file.buffer,
    file.mimetype, file.originalname);
}
```

Inject `ImportInvoiceUseCase` into the controller directly (or add a method to `InvoicesService` — prefer direct injection to keep it simple since `InvoicesService` is already injected there).

## Step 9 — InvoicesModule: add providers and imports

`src/modules/Finance/Invoices/invoices.module.ts`

```
imports: [
  forwardRef(() => PaymentModule),
  FinancialEventsModule, // new — provides FinancialEventsService
  LedgerModule,          // new — provides LedgerEngine
],
```

```
providers: [
  ...,
  ImportInvoiceUseCase,    // new
  ExpensesRepository,      // new – for creating expenses in import flow
],
```

`ExpensesRepository` can be added as a local provider because `MySQLDatabase` and `AuthContextService` are globally provided ( `@Global()` on `DatabaseModule` ; `AuthContextService` is request-scoped and globally available).

## PR 2 — Proteção de Mutação de Conta Bancária/Cartão

### Architecture

```
UpdateExpense.execute() / UpdateRevenue.execute()
|
├ (if idBankAccount or idCreditCard changed vs current)
|   |
|   └ FinancialEventsService.hasEventForEntity(referenceType, entityId)
|       |
|       └ FinancialEventsRepository.hasEventForEntity(...)
|
└ if true → throw Error (cannot change account after event recorded)
```

### Step 1 — DB Migration: Index on `financial_events`

New file:

`src/migrations/versions/Migration20260410_03_IndiceFinancialEventsReference.ts`

```
ALTER TABLE financial_events
  ADD INDEX idx_reference (clientId, referenceType, referenceId);
```

### Step 2 — `FinancialEventsRepository`: add `hasEventForEntity`

`src/modules/Finance/core/financial-events/financial-events.repository.ts`

```
async hasEventForEntity(referenceType: PaymentType, referenceId: number):
Promise<boolean> {
  const query = `SELECT 1 FROM financial_events
                WHERE clientId = ? AND referenceType = ? AND referenceId = ?
LIMIT 1`;
  const result = await this.database.select(query, [
    this.authContext.getClientId(), referenceType, referenceId
  ]);
  return result.length > 0;
}
```

### Step 3 — `FinancialEventsService`: delegate method

`src/modules/Finance/core/financial-events/financial-events.service.ts`

```
async hasEventForEntity(referenceType: PaymentType, referenceId: number):  
Promise<boolean> {  
    return this.repository.hasEventForEntity(referenceType, referenceId);  
}
```

## Step 4 — UpdateExpense: account-change guard

`src/modules/Finance/Expenses/UseCases/UpdateExpense.ts`

In `execute()` , before saving, add:

```
const current = await this.repository.getExpenseById(id);  
if (!current) throw new Error('Expense not found');  
  
const accountChanged =  
    (expense.idBankAccount ?? 0) !== current.idBankAccount ||  
    (expense.idCreditCard ?? 0) !== current.idCreditCard;  
  
if (accountChanged) {  
    const hasEvent = await  
this.financialEventsService.hasEventForEntity(PaymentType.EXPENSE, id);  
    if (hasEvent) {  
        throw new Error('Conta bancária não pode ser alterada após registro de evento  
financeiro');  
    }  
}
```

Inject `FinancialEventsService` into `UpdateExpense` constructor.

## Step 5 — UpdateRevenue: account-change guard

`src/modules/Finance/Revenues/UseCases/UpdateRevenue.ts`

Same pattern, checking only `idBankAccount` (revenue has no `idCreditCard`):

```
const current = await this.repository.getRevenueById(id);  
if (!current) throw new Error('Revenue not found');  
  
if ((revenue.idBankAccount ?? 0) !== current.idBankAccount) {  
    const hasEvent = await  
this.financialEventsService.hasEventForEntity(PaymentType.REVENUE, id);  
    if (hasEvent) {  
        throw new Error('Conta bancária não pode ser alterada após registro de evento  
financeiro');  
    }  
}
```

Inject `FinancialEventsService` into `UpdateRevenue` constructor.

## Step 6 — Module wiring

`src/modules/Finance/Expenses/expenses.module.ts` — add `FinancialEventsModule` to imports, add `FinancialEventsService` to `UpdateExpense`'s visible providers (auto-resolved via module import).

`src/modules/Finance/Revenues/revenue.module.ts` — add `FinancialEventsModule` to imports.

Both modules currently have no `FinancialEventsModule` import.

---

## Critical Files Summary

| File                                                                                  | Action                                                                                                                                   | PR |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|----|
| <code>src/enum/financial-events.enum.ts</code>                                        | Add <code>INVOICE_RECOGNIZED</code>                                                                                                      | 1  |
| <code>src/modules/Finance/core/ledger/ledger.builder.ts</code>                        | Handle <code>INVOICE_RECOGNIZED</code> in <code>defineAmounts</code>                                                                     | 1  |
| <code>src/EntityModels/CreditCard.ts</code>                                           | Add <code>paymentAccountId</code> field                                                                                                  | 1  |
| <code>src/Shared/interfaces/ICreditCardRow.ts</code>                                  | Add <code>paymentAccountId?</code>                                                                                                       | 1  |
| <code>src/modules/Finance/credit-cards/DTOs/credit-card.dto.ts</code>                 | Add <code>paymentAccountId?</code>                                                                                                       | 1  |
| <code>src/modules/Finance/Invoices/invoices.repository.ts</code>                      | Add <code>getCreditCardForImport</code> , <code>findInvoiceByDueDate</code>                                                              | 1  |
| <code>src/modules/Finance/Invoices/invoices.controller.ts</code>                      | Add <code>POST /invoices/import</code>                                                                                                   | 1  |
| <code>src/modules/Finance/Invoices/invoices.module.ts</code>                          | Add <code>FinancialEventsModule</code> , <code>LedgerModule</code> , <code>ExpensesRepository</code> , <code>ImportInvoiceUseCase</code> | 1  |
| <code>src/modules/Finance/core/financial-events/financial-events.repository.ts</code> | Add <code>hasEventForEntity</code>                                                                                                       | 2  |
| <code>src/modules/Finance/core/financial-events/financial-events.service.ts</code>    | Add <code>hasEventForEntity</code> delegate                                                                                              | 2  |
| <code>src/modules/Finance/Expenses/UseCases/UpdateExpense.ts</code>                   | Add account-change guard                                                                                                                 | 2  |
| <code>src/modules/Finance/Revenues/UseCases/UpdateRevenue.ts</code>                   | Add account-change guard                                                                                                                 | 2  |
| <code>src/modules/Finance/Expenses/expenses.module.ts</code>                          | Import <code>FinancialEventsModule</code>                                                                                                | 2  |
| <code>src/modules/Finance/Revenues/revenue.module.ts</code>                           | Import <code>FinancialEventsModule</code>                                                                                                | 2  |

New migrations: `Migration20260410_01` , `_02` , `_03`

New import dir: `src/modules/Finance/Invoices/import/` (10 files)

---

## Verification

### PR 1

1. `POST /invoices/import` with a CSV (3 rows, same closing cycle) → 1 invoice, 3 expenses, 1 `INVOICE_RECOGNIZED` in `financial_events` with `accountId = paymentAccountId` ,



`referenceId = invoice.id`

2. Same file with rows spanning two closing cycles → 2 invoices, 3 expenses, 2 `INVOICE_RECOGNIZED` events
3. CSV upload with `paymentAccountId = NULL` on the card → HTTP 400 with descriptive message, no DB writes
4. Malformed CSV (bad date, non-numeric amount) → HTTP 400, no transaction committed
5. OFX file (exported from a bank) → same validation as CSV
6. Check `ledger_entries` : each `INVOICE_RECOGNIZED` event should produce 2 entries (INVOICE + account type)

## PR 2

1. Create expense with `idBankAccount = X` → fire a payment (creates `financial_events` record) → `PATCH /expenses/:id` with different `idBankAccount` → HTTP 500 with "Conta bancária não pode ser alterada"
2. Create expense with `idBankAccount = X` but **no payment yet** → `PATCH` with different `idBankAccount` → succeeds
3. Alter other fields (name, value) on a paid expense → succeeds
4. Create revenue → pay it → attempt to change `idBankAccount` → blocked
5. `SHOW INDEX FROM financial_events` → `idx_reference` present